

Learning-Guided Simulated Annealing for the Capacitated Vehicle Routing Problem

Anonymous submission

Abstract

Introduction

Background and Related Work

Problem Definition

The *Capacitated Vehicle Routing Problem (CVRP)* is defined on a complete undirected graph $G = (V, E)$, where $V = \{0\} \cup \mathcal{C}$ and $E = V \times V$ denote the sets of nodes and edges, respectively. Node 0 corresponds to the central depot, and $\mathcal{C} = \{1, \dots, N\}$ represents the set of N customers.

Each edge $(i, j) \in E$ is associated with a travel cost $c_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\|_2$, the Euclidean distance between the 2D coordinates $\mathbf{p}_i, \mathbf{p}_j \in [0, 1]^2$, satisfying $c_{ij} = c_{ji}$. Each customer $i \in \mathcal{C}$ has a non-negative demand q_i , while the depot has zero demand ($q_0 = 0$). An unlimited fleet of identical vehicles, each with uniform capacity Q , is stationed at the depot; the number of routes m is therefore a decision variable unconstrained from above.

A route $R_k = (0, v_1, v_2, \dots, v_p, 0)$ is a simple cycle starting and ending at the depot, where $\{v_1, \dots, v_p\} \subseteq \mathcal{C}$ denotes the subset of customers served by vehicle k . A *solution* $s = \{R_1, \dots, R_m\}$ consists of a set of m routes. A solution s is *feasible* if and only if it satisfies the following conditions:

1. Each customer is visited exactly once by a single vehicle,

$$\sum_{k=1}^m \mathbf{1}_{\{i \in R_k\}} = 1, \quad \forall i \in \mathcal{C}.$$

2. For every route R_k , the total demand of its customers does not exceed the vehicle capacity Q , i.e.,

$$\sum_{i \in R_k \setminus \{0\}} q_i \leq Q, \quad \forall k = 1, \dots, m.$$

The set of *feasible solutions* is denoted by $\mathcal{F}$.

Let $x_{ij}(s)$ be a binary indicator variable with $x_{ij}(s) = 1$ if edge (i, j) is traversed in any route R_k of s , and $x_{ij}(s) = 0$

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

otherwise. The objective of the CVRP is to find a feasible solution $s^* \in \mathcal{F}$ that minimizes the total travel cost $C(s)$:

$$s^* = \arg \min_{s \in \mathcal{F}} \left\{ C(s) = \sum_{i \in V} \sum_{j \in V, i < j} c_{ij} x_{ij}(s) \right\}.$$

Proposed Method

Overview

We propose *Learning-Guided Simulated Annealing (LG-SA)*, a hybrid method that combines *Simulated Annealing (SA)* with a lightweight neural policy trained by *Reinforcement Learning (RL)*. Following the Neural Simulated Annealing framework (Correia, Worrall, and Bondesan 2023), the SA component orchestrates the global exploration of the solution space through its acceptance criterion and cooling schedule, while the neural network replaces the single blind component of SA — the uniform move proposal — with a learned, state-dependent proposal distribution. This design contrasts with “heavyweight” improvement architectures such as the Dual-Aspect Collaborative Transformer (Ma et al. 2021), which replace the heuristic logic entirely with a deep encoder: our policy remains deliberately small, so that the guided search retains a per-step cost close to that of classical SA and can be batched over thousands of instances on a single GPU.

Simulated Annealing (Kirkpatrick, Gelatt, and Vecchi 1983) iteratively explores the neighborhood of a current solution. Given a solution s , SA samples a neighboring solution $s' \in \mathcal{N}(s)$ and accepts it according to the *Metropolis criterion*: improving moves are always accepted, while worsening moves are accepted with probability $\exp(-\Delta C/T)$, where $\Delta C = C(s') - C(s)$ is the cost difference and T the current temperature. The temperature decays from an initial value T_0 to a final value T_{final} over the search horizon according to a monotone cooling schedule, which regulates the transition from exploration to exploitation.

Neighboring solutions are produced by a *neighborhood operator* $H : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ that maps a solution s and an action $a \in \mathcal{A}(s)$ to the neighbor $s' = H(s, a)$, so that

$$\mathcal{N}(s) = \{H(s, a) : a \in \mathcal{A}(s)\}.$$

In our setting, an action designates a pair of nodes $a = (i, j)$, and H applies a standard pairwise local search move to them (e.g., relocating node i next to node j). The operator is a

Algorithm 1 Learning-Guided Simulated Annealing

Input: instance, initial solution s , policy π_θ , operator H **Output:** best solution found s^*

```
1: Initialize  $s^* \leftarrow s$  and temperature  $T \leftarrow T_0$ 
2: while termination criterion not met do
3:   Encode  $s$  into per-node feature vectors
4:   Sample action  $a \sim \pi_\theta(\cdot | s)$  (guided proposal)
5:   Generate candidate  $s' \leftarrow H(s, a)$ 
6:    $s \leftarrow s'$  with probability  $\min(1, e^{-\Delta C/T})$  (Metropolis)
7:   Update  $s^*$  if  $C(s) < C(s^*)$ ; cool  $T$ 
8: end while
9: return  $s^*$ 
```

modular component of the framework; the candidate operators and their comparison are presented in the experimental section.

Whereas classical SA draws a uniformly over $\mathcal{A}(s)$, LG-SA samples it from a policy π_θ conditioned on the current solution:

$$a \sim \pi_\theta(\cdot | s) \quad \text{and} \quad s' = H(s, a) \in \mathcal{N}(s).$$

Everything else — the operator, the Metropolis acceptance, the cooling schedule — is kept from classical SA. The overall procedure is summarized in Algorithm 1 and illustrated in Figure 1.

The Guided Search as a Markov Decision Process

To learn the proposal distribution with RL, we frame the SA loop as a Markov Decision Process. At step t , the *state* s_t is the current feasible solution together with the search meta-information (current temperature and remaining budget), encoded as a set of per-node feature vectors described below. The *action* $a_t = (i, j)$ is the node pair handed to the operator H . The *transition* is the composition of the operator and the stochastic Metropolis acceptance: the next state is $H(s_t, a_t)$ if the move is accepted and s_t otherwise, so the environment dynamics include the acceptance randomness of SA.

The *reward* is derived from the evolution of the solution cost along the search: a move that is accepted and improves the solution receives a positive reward proportional to the (normalized) cost reduction, a rejected move yields no improvement signal, and proposals leading to infeasible neighbors can be penalized. Reward variants that instead emphasize improvements of the best solution found so far, or the final solution quality at the end of the horizon, fit the same framework; the precise shaping is a design choice studied experimentally. The agent thus maximizes the expected cumulative discounted reward over the SA horizon, which aligns the learned proposal with the overall goal of driving the annealing process toward low-cost solutions rather than with any single greedy step.

Node-Centric State Representation

To be applicable to any problem size N and to remain *scale-invariant*, the state is encoded in a *node-centric* fashion:

each node i carries its own local feature vector $\mathbf{x}_i$, and no fixed-size global descriptor is used. The features fall into four groups:

- *Static instance features*: node coordinates, polar coordinates relative to the depot, a depot indicator, and the normalized demand q_i/Q ;
- *Solution topology and local geometry*: the coordinates of the node’s current predecessor and successor in its route, together with local quality signals such as the detour cost incurred by the node at its current position, its distance to the route centroid, and local density statistics of the instance;
- *Capacity status*: the load ratio and remaining slack of the node’s current route, and the node’s own contribution to that load;
- *Search meta-features*: the current (normalized) SA temperature and the fraction of the search budget remaining, which let the policy adapt its behavior to the phase of the annealing schedule.

All features are local to a node and standardized, so the same trained model can be deployed on instances of arbitrary size without retraining. The complete list of features, with their formal definitions, is given in the appendix; their individual contribution is analyzed in the experimental section.

Two-Stage Conditional Policy

The policy must define a distribution over the $\mathcal{O}(N^2)$ node pairs while remaining cheap enough to be evaluated at every SA step. We therefore factorize the joint selection as

$$P(i, j | s) = P(i | s) \cdot P(j | i, s),$$

and implement it with a lightweight two-stage architecture, illustrated in Figure 2. In the first stage, a small network f_θ maps each feature vector to a scalar score $z_i = f_\theta(\mathbf{x}_i)$, and the first node is sampled from the softmax distribution over these scores:

$$i \sim P(\cdot | s) = \text{softmax}(\{z_i : i \in \mathcal{C}\}).$$

In the second stage, the representation of the selected node i is combined with that of every other node j , and a second network g_θ assigns a conditional score $z_{ij} = g_\theta(\mathbf{x}_i, \mathbf{x}_j)$ from which the partner is sampled:

$$j \sim P(\cdot | i, s) = \text{softmax}(\{z_{ij} : j \in \mathcal{C}\}).$$

Both stages are built from small multilayer perceptrons applied identically to every node, which makes the policy permutation-equivariant. The second stage can additionally be conditioned on cheap pairwise descriptors of the candidate move (e.g., the exact cost of the move joining i and j), computed only for the selected i .

This factorization also provides a natural hook for the capacity constraints: once i is selected, the partners j for which $H(s, (i, j))$ would be infeasible are masked out of the second-stage distribution, so every proposed neighbor is feasible at the price of $\mathcal{O}(N)$ feasibility checks per step. Alternative constraint-handling strategies are discussed and compared in the experimental section.

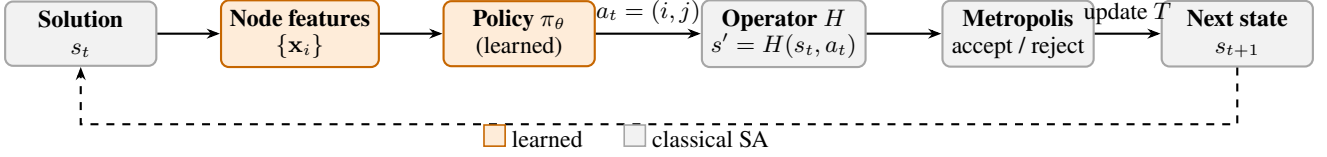


Figure 1: The LG-SA loop. Classical SA components (gray) are kept unchanged: the operator H , the Metropolis acceptance, and the cooling schedule. The only modified block is the move proposal (orange): the current solution is encoded into per-node feature vectors and a learned policy samples the node pair $a_t = (i, j)$, replacing the uniform sampling of classical SA.

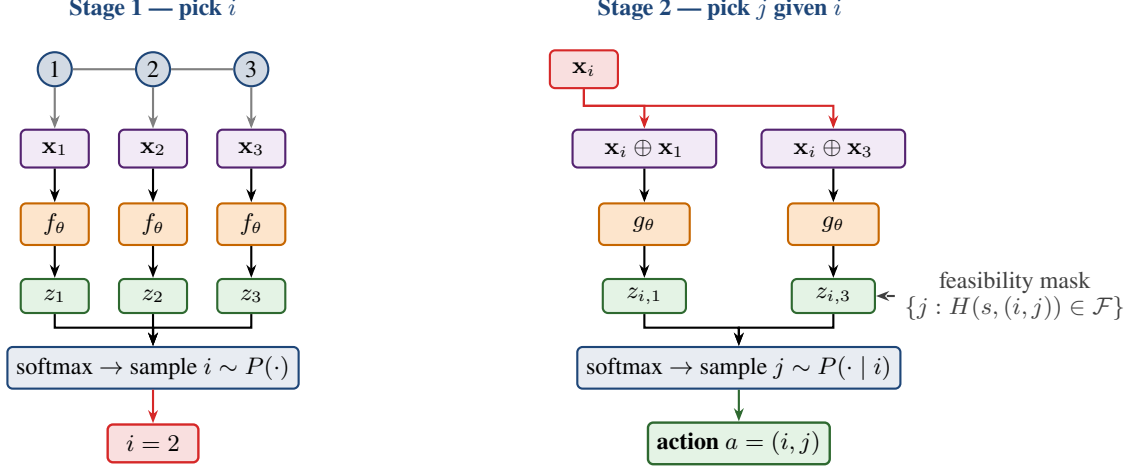


Figure 2: Two-stage conditional sampling of the node pair (i, j) , shown on a toy route of three customers. Stage 1: every node’s feature vector is scored independently by a shared network f_θ and the first node i is sampled from the softmax over the scores. Stage 2: the representation of the selected node i is combined with that of every candidate partner j and scored by a second network g_θ ; infeasible partners are masked out before the softmax, and j is sampled from the conditional distribution. The factorization $P(i, j) = P(i) \cdot P(j | i)$ makes each proposal $\mathcal{O}(N)$.

Since the conditional network is evaluated only for the single selected first node, each proposal costs $\mathcal{O}(N)$ — as opposed to the $\mathcal{O}(N^2)$ cost of scoring all pairs jointly — and the whole step is expressed as batched tensor operations, allowing thousands of instances to be advanced in parallel on one GPU.

Training with PPO

The policy is trained with *Proximal Policy Optimization* (PPO) (Schulman et al. 2017), the standard on-policy actor-critic algorithm. Training alternates between a *collection* phase, in which the current policy runs the full LG-SA loop of Algorithm 1 on a large batch of freshly generated instances in parallel and stores every transition (s_t, a_t, r_t) , and an *update* phase, in which the stored episodes are replayed for several optimization epochs. Advantages are estimated with Generalized Advantage Estimation, and the actor maximizes the usual clipped surrogate objective with an entropy bonus; we refer to Schulman et al. (2017) for the standard formulation and only detail our departures from it.

The critic V_ϕ is a separate permutation-invariant network: it encodes each node feature vector independently, pools the embeddings across nodes into a fixed-size summary, and maps this summary to a scalar value estimate. Keeping the

actor and critic distinct prevents interference between the policy and value representations.

Our implementation departs from standard PPO in three ways, all aimed at stabilizing training on the long and noisy SA horizon:

- *Hybrid clipping–KL objective.* Rather than choosing between the clipped surrogate and the KL-penalized objective, we use both: an adaptive Kullback–Leibler penalty $\beta_{\text{KL}} D_{\text{KL}}(\pi_{\theta_{\text{old}}} || \pi_\theta)$ is added to the clipped loss, and its coefficient is doubled or halved after each update epoch depending on whether the measured divergence leaves a band around a target value. This soft trust region allows many optimization epochs on each collected batch without catastrophic policy drift.
- *KL-coupled learning rate.* The same divergence signal modulates the actor’s learning rate within fixed bounds — gently decreased when the policy moves too fast, increased when it stalls — so the effective step size tracks the trust region automatically.
- *Regularized critic.* The value loss is clipped, mirroring the actor’s trust region, and augmented with a gradient penalty that encourages a smooth (near-Lipschitz) value landscape; we found this useful given that consecutive

states differ by a single local move whose acceptance is itself stochastic.

Experimental Results

Experimental Setup

Benchmark. All experiments reported in this section use the uniform benchmark distribution of Nazari et al. (2018): depot and customer coordinates drawn uniformly in $[0, 1]^2$, demands drawn uniformly in $\{1, \dots, 9\}$, and a size-dependent vehicle capacity ($Q = 20, 30, 40, 50$ for $N = 10, 20, 50, 100$). For each size we pre-generate a fixed test set of 10,000 instances that is reused by every configuration, so that all comparisons are paired at the instance level. Unless stated otherwise, results are reported at $N = 100$.

Evaluation protocol. A trained model is evaluated by running the LG-SA loop over the full test set in parallel on a single GPU, starting from random feasible solutions and running 10,000 SA steps per instance with half-precision inference. We report the mean final routing cost over the 10,000 instances and the wall-clock time of the whole batch. Classical (blind) SA — uniform proposals with the same operator, step budget, and cooling schedule — is evaluated under the identical protocol, as is an OR-Tools baseline with a fixed per-instance time limit (Furnon and Perron 2024).

Statistical protocol. Every configuration is trained under 3 random seeds (5 for confirmations and headline runs); the objective is the mean final cost over the test set, averaged over seeds and denoted $\bar{c}$. Comparisons are *seed-paired*: competing configurations share their training seeds, and we report the mean and standard deviation of the per-seed differences $\Delta = \bar{c}(\text{variant}) - \bar{c}(\text{reference})$, together with their sign consistency. Significance is judged against a per-batch *noise floor* σ_{floor} — the pooled seed-to-seed standard deviation of the stable arms of the batch at hand — rather than against a single global threshold, because the seed variance is itself configuration-dependent: unstable arms (e.g., sparse rewards or structured initializations) inflate the pooled spread and would mask real effects measured elsewhere. An effect is called significant only if $|\Delta| > \sigma_{\text{floor}}$; below the floor it is reported as a tie.

Sequential validation protocol. The design space of LG-SA is an ordered tuple of axes $\mathcal{A} = (a_1, \dots, a_K)$ — operator and feasibility strategy, node features, architecture, reward, annealing parameters, acceptance, initialization, optimization budget, curriculum — each axis a_k ranging over a finite candidate set $\mathcal{C}_k$. This space is far too large for a full factorial study, so we validate it by coordinate descent: the axes are swept one at a time in order of expected leverage, and the winner of each sweep is frozen into the configuration before the next axis is examined, so later sweeps inherit all earlier decisions. Targeted follow-up batches probe the interactions we consider most at risk, and the complete winning configuration is retrained from scratch with 5 seeds as a final confirmation.

Axis	Adopted setting	#	Effect	App.
Operator $\times$ feasibility	insertion + masking	9	+0.29**	B
Feature set	all 13 groups	6	+0.34**	C
Pair descriptors	none	4	+0.14*	C
Critic	multi-statistic	2	+0.06**	D
Actor capacity	32×1	6	tie	D
Actor structure	indep. scorers	6	+0.51*	D
Distribution shaping	logit clip $C=10$	4	+0.17**	D
Reward	immediate	9	tie	E
Learning rates	$3 \times 10^{-4} / 10^{-3}$	9	+0.05*	F
Entropy coefficient	0.01	3	+0.13**	F
Rollout length	200	3	tie	F
Cooling shape	geometric	2	tie	E
Temperature range	$T_0=1, T_{\text{final}}=0.01$	6	+0.17**	E
Metropolis	on (train + test)	4	+0.12**	E
Training construction	random	5	+0.22**	E
Combined confirmation rates, rollout, T_{final}		—	+0.22**	F

Table 1: Master summary of the design validation, one row per axis in the paragraph order of this subsection. *Effect*: seed-paired margin (routing-cost units, two decimals) in favor of the adopted setting — measured against the previous incumbent when the sweep changed the configuration, and against the strongest meaningfully distinct alternative when it confirmed it; “tie” when no alternative cleared that batch’s noise floor. *: $|\Delta| > \sigma_{\text{floor}}$; **: $|\Delta| > 2\sigma_{\text{floor}}$ (per-batch floors in the appendices). #: candidate settings compared on the axis (diagnostic-only runs not counted). The rollout length is a standalone tie adopted for its contribution to the combined configuration, whose joint retrain (bottom row) confirms the adopted optimization changes are additive.

Design Choices

Table 1 summarizes the outcome of the coordinate descent: fifteen design axes and 78 candidate settings, validated under the common protocol above. The paragraphs below state what was adopted on each axis and why; the per-axis candidate definitions, full grids, and per-seed diagnostics are collected in Appendices B–F.

Search operators. The first sweep jointly fixes the operator H and the constraint-handling strategy: the three pairwise operators (swap, insertion, 2-opt/2-opt*) are crossed with three ways of keeping the search feasible — *proactive masking* (the second-stage mask of Figure 2), *reactive rejection* of infeasible neighbors, and an *indirect representation* that repacks an unconstrained permutation into feasible routes at every step (definitions in Appendix B). Insertion with masking dominates the grid (Table 2): lowest cost, lowest seed variance, and among the cheapest cells, with both factors also winning marginally. Relocating a single node is the move that best matches a learned pointer to a node pair, whereas rejection is uniformly the worst strategy — discarded proposals waste both search budget and learning signal. The decision is robust: the runner-up (+0.29, insertion with the indirect representation) points to the same operator.

Node representation. Single-group diagnostics mislead on this axis. Leave-one-out and isolation protocols (Table 7,

Operator	Masking	Indirect	Rejection
Insertion	16.68 ± 0.03	16.97 ± 0.04	26.36 ± 0.71
Swap	19.50 ± 1.54	22.82 ± 1.36	35.75 ± 3.04
2-opt(*)	18.93 ± 0.09	21.29 ± 0.39	25.84 ± 0.42

Table 2: Operator $\times$ feasibility study: mean final cost $\pm$ seed std at $N = 100$ (10^4 SA steps, 10,000 instances, 3 seeds; all runs start from mean cost 57.87). Insertion with proactive masking wins and is also the most stable cell. Definitions, main effects, compute, and per-seed diagnostics in Appendix B.

Set	Groups	Cost	Time (s)
full set (adopted)	13	16.683	173
no load ratio	12	16.648	173
set7	7	17.020	129
nodepct6	6	17.088	121
centroid6	6	17.100	122
core5	5	17.162	115

Table 3: Best-set validation at 5 seeds, seed-paired against the full set ($\sigma_{\text{floor}} \approx 0.057$). *core5* = static, meta, topology, neighbor ranks, detour; *set7* adds centroid and load share. Every pruned set is significantly worse; the only quality-neutral trim is dropping the load ratio alone. Paired deltas and per-seed views in Appendix C.

appendix) flag only six of the 13 feature groups of Table 4 as individually useful, but retraining the candidate minimal sets head-to-head at 5 seeds reverses the pruning rule (Table 3): every pruned set is significantly worse, degrading monotonically as groups are removed — the smallest penalty ($+0.337 \pm 0.091$, set7) is already $6\times$ the floor of 0.057 — so the density and capacity groups that look redundant one at a time carry real joint value. The only quality-neutral trim is dropping the route load ratio (a tie, -0.035 ± 0.106); pruning otherwise buys compute, not quality (set7 trades 2% quality for a 25% speedup). The two conditional pair descriptors of the second stage *hurt* when enabled ($+0.14 \pm 0.11$ and $+0.18 \pm 0.07$, additive, at ≈ 20 s extra wall-clock each): the per-node features already encode the local geometry they summarize. The adopted representation is the full 13-group set with both descriptors off (full diagnostics in Appendix C).

Policy and learning. Nothing that enlarges or rewires the actor pays for itself: the capacity grid is a plateau (no cell clears the 0.053 floor, at up to $+38\%$ wall-clock), appending a pooled instance summary to the scoring stages worsens every seed ($+0.508 \pm 0.496$, $+48\%$ wall-clock), and forcing both selection stages through a shared encoder is a cliff ($+3.20$ for its best cell, $18\times$ that batch’s floor of 0.176). The two adopted changes add no actor capacity: the multi-statistic critic (-0.058 , twice the 0.027 floor, and free at deployment since the critic is discarded after training) and logit clipping at $C = 10$ (-0.173 ± 0.059 against a floor of 0.072, unanimous, at unchanged compute) — at $N = 100$ the leverage lies in the quality of the value estimate and in keeping the proposal distribution from collapsing, exactly

the failure mode a near-deterministic policy induces inside a Metropolis loop (Appendix D). The reward only needs to be *dense*: the six variants built on per-step improvement tie within a band 0.07 wide, while removing per-step credit is catastrophic — the terminal reward loses 2.95 and the step-penalty schemes 0.7 to 1.1, all with unstable training (Appendix E). Among the optimizer knobs, the actor learning rate is the axis that matters (monotone across the 3×3 grid, main effect ≈ 0.05 per grid step against a floor of 0.031), the entropy coefficient confirms its default at the bottom of a U (0 costs $+0.128 \pm 0.021$, 0.05 costs $+0.783 \pm 0.036$), and the rollout length is asymmetric: halving it is a large regression ($+0.444 \pm 0.015$), doubling it a marginal, sign-split gain adopted for the combined configuration. Retrained jointly, the adopted changes are additive (-0.223 ± 0.016 against the pre-sweep incumbent, unanimous — more than any single axis delivered) after one reproducible divergence pinned the critic rate at 10^{-3} : a critic stepping $10\times$ faster than the actor overshoots on the sparse advantages that deep cooling produces early in training (mechanism in Appendix F).

Annealing search. The cooling *shape* is a tie (geometric against a piecewise-constant step schedule: -0.013 ± 0.034 against a floor of 0.044, sign split), but the *range* holds the largest single-knob gain of the validation: over a 3×2 grid of endpoints, every $T_{\text{final}} = 0.01$ cell beats every $T_{\text{final}} = 0.1$ cell, worth -0.165 ± 0.034 at the incumbent T_0 ($4\times$ the 0.041 floor, unanimous at every starting temperature, which is itself inert) — the low-temperature endgame is precisely where the learned proposal performs its fine-grained refinement, and a high floor cuts that phase off early. The Metropolis criterion earns its keep at deployment, not at training: in a 2×2 train/test cross, removing the acceptance at test time costs $+0.119 \pm 0.025$ ($3.5\times$ the 0.034 floor, unanimous, whichever way the actor was trained), while the training-time flag is a wash and stays on. For the same reason the solver keeps *sampling* $a_t \sim \pi_\theta(\cdot | s_t)$ at inference rather than taking the greedy argmax: a near-deterministic proposal would collapse the diversity the acceptance step needs. Finally, the starting solution matters exactly where intuition says it should not: at deployment, 10^4 annealing steps wash out even a $6\times$ spread in construction cost (final band 0.160 wide across five constructions), but at training time random constructions are decisive — actors trained from structured heuristics lose 1.1 to 2.8 at every deployment, and even mixing constructions dilutes coverage ($+0.222 \pm 0.084$ against the 0.063 floor) — because starting from the worst solutions exposes the policy to the widest state distribution (Appendix E).

Curriculum. Training episodes are far shorter than the search budgets used at deployment, so a policy trained only from raw constructive solutions rarely visits the fine-grained regime near local optima that dominates the end of a long search. The curriculum closes this gap by moving the *starting point*: before each collection phase, the batch of instances is first warmed up by the current policy for t_{init} steps, and t_{init} ramps over training from 0 to a maximum warm-up strength ξ_{CL} over E epochs (Figure 3) — early training repairs poor solutions, late training refines already-improved ones. Two alternative mechanisms place the same idea elsewhere: a per-

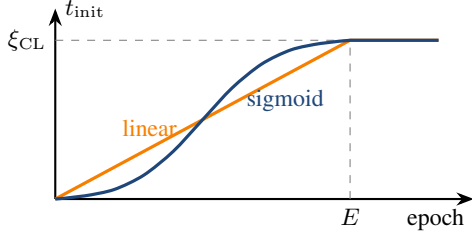


Figure 3: Curriculum ramp: the warm-up depth t_{init} grows from 0 to the warm-up strength ξ_{CL} over E training epochs, following a linear or sigmoid schedule, and stays saturated afterwards.

instance random depth $d_i \sim \mathcal{U}\{0, \dots, t_{\text{init}}\}$, which makes a single batch cover the whole spectrum from raw to deeply improved solutions, and *persistent chains*, where instances and their best solutions survive across epochs and the search resumes from them (a fraction of chains being refreshed with new instances each epoch), obtaining deep starting points at no extra warm-up compute. The three mechanisms are validated against a common 5-seed no-curriculum baseline that also provides this batch’s noise floor, followed by a budget-matched coverage ablation and a 5-seed confirmation with out-of-distribution evaluations.

Final Model and Comparison with Baselines

Final model. All component winners of Table 1, the best curriculum, and the tuned optimization budget are combined into the final model, trained with 5 seeds; the best checkpoint provides the headline numbers. We compare it against blind SA under matched conditions (same operator, budget, and schedule) and against the OR-Tools baseline, and we report the mean cost as a function of the SA step budget from 10^2 to 10^6 steps to characterize the anytime behavior of the guided search.

Generalization and scalability. Finally, we probe how far the node-centric representation carries across problem scales: models trained at each size are cross-tested at every other size, and the final model is evaluated on instances well beyond its training size with the same checkpoint, reporting both solution quality and runtime scaling.

Conclusion

References

- Correia, A. H. C.; Worrall, D. E.; and Bondesan, R. 2023. Neural Simulated Annealing. In *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, 4946–4962. PMLR. ISSN: 2640-3498.
- Furnon, V.; and Perron, L. 2024. OR-Tools Routing Library.
- Kirkpatrick, S.; Gelatt, C. D.; and Vecchi, M. P. 1983. Optimization by Simulated Annealing. 220.
- Kool, W.; van Hoof, H.; and Welling, M. 2019. Attention, Learn to Solve Routing Problems! In *International Conference on Learning Representations*.

Ma, Y.; Li, J.; Cao, Z.; Song, W.; Zhang, L.; Chen, Z.; and Tang, J. 2021. Learning to Iteratively Solve Routing Problems with Dual-Aspect Collaborative Transformer. In *Advances in Neural Information Processing Systems*, volume 34, 11096–11107. Curran Associates, Inc.

Nazari, M.; Oroojlooy, A.; Snyder, L.; and Takac, M. 2018. Reinforcement Learning for Solving the Vehicle Routing Problem. In *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. ArXiv:1707.06347 [cs].

Vidal, T. 2016. Technical note: Split algorithm in $O(n)$ for the capacitated vehicle routing problem. *Computers & Operations Research*, 69: 40–47.

Appendix Overview

The appendices mirror the order of the design axes in the main text. *State Feature Definitions* gives the formal definition of every component of the node feature vector. The five *Full Results* appendices then carry the evidence behind the Design Choices subsection — for operators and feasibility, features, architecture, the search-loop components (reward, annealing, acceptance, initialization), and the optimization hyperparameters (including the per-seed record and mechanism of the combined-configuration divergence): the per-axis motivation and candidate definitions, the complete grids and secondary tables with paired deltas and per-batch noise floors, and the per-seed diagnostic plots behind the master summary (Table 1) and the two headline tables. Two conventions are shared throughout: every comparison is seed-paired (both arms are trained on the same seeds), and every delta plot shows the per-seed differences against a shaded band marking that batch’s noise floor — an effect is trusted only if it clears the band with a consistent sign across seeds. Each full-results appendix opens with the mapping from the internal configuration names appearing in the plot legends to the paper terminology.

Appendix A: State Feature Definitions

This appendix defines every component of the node feature vector $\mathbf{x}_i$ used by the policy and the critic. Throughout, $\text{prev}(i)$ and $\text{succ}(i)$ denote the nodes immediately before and after i in the current visiting order, $r(i)$ the route currently serving i , $Q_{r(i)}$ the total load of that route, and $d(\cdot, \cdot)$ the Euclidean distance. Figure 4 illustrates the geometric quantities, and Table 4 summarizes the groups and their dimensions. All features are normalized to comparable ranges, and each is computed locally at node i : the encoding contains no fixed-size global descriptor, which is what makes the representation applicable to any instance size.

Static instance features (6). The 2D coordinates $\mathbf{p}_i = (p_i^x, p_i^y) \in [0, 1]^2$; a depot indicator $\mathbf{1}_{\{i=0\}}$; the polar angle θ_i of the node around the depot; the distance to the depot d_{i0} , min–max normalized per instance; and the demand normalized by the vehicle capacity, q_i/Q . These features depend only on the instance, not on the current solution.

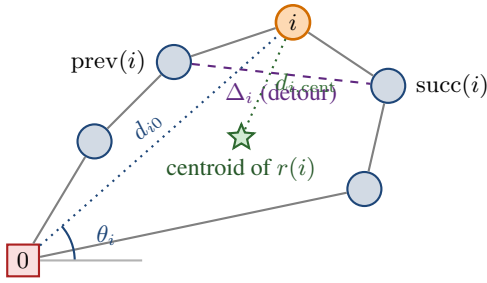


Figure 4: Geometric features attached to a node i (orange) within its current route. The detour Δ_i measures the extra length caused by visiting i between its neighbors; (θ_i, d_{i0}) locate i in polar coordinates around the depot; $d_{i,\text{cent}}$ measures how far i lies from the center of gravity of its own route.

Route topology (4). The coordinates $\mathbf{p}_{\text{prev}(i)}$ and $\mathbf{p}_{\text{succ}(i)}$ of the node’s current neighbors in the visiting order. They give the policy direct access to the local shape of the route around each node.

Neighbor distance-ranks (2). For each of $\text{prev}(i)$ and $\text{succ}(i)$, the rank of that neighbor in the list of all nodes sorted by distance from i , normalized to $[0, 1]$ (the nearest node has rank ≈ 0). A well-placed node is linked to low-rank neighbors; a high rank signals a locally poor connection, independently of the absolute scale of the instance.

Relative demand (1). The demand normalized by the largest demand in the instance, $q_i / \max_{j \in C} q_j$, complementing q_i / Q with a signal that is invariant to the capacity level.

Spatial density (3). The mean distance from i to its k nearest neighbors, evaluated at three scales: $k = 5$, $k = \lceil N/10 \rceil$, and $k = \lceil N/3 \rceil$. These static statistics distinguish nodes in dense clusters from isolated ones.

Local solution quality (2). The *detour cost*

$$\Delta_i = d(\text{prev}(i), i) + d(i, \text{succ}(i)) - d(\text{prev}(i), \text{succ}(i)),$$

i.e., the extra length the tour pays for visiting i at its current position (equivalently, the saving obtained by removing it); and the distance $d_{i,\text{cent}}$ from i to the centroid of the nodes of its route, which flags spatial outliers assigned to the wrong route.

Capacity status (3). The load ratio of the node’s route, $L_{r(i)} = Q_{r(i)} / Q$; the remaining slack, $S_{r(i)} = (Q - Q_{r(i)}) / Q$; and the node’s share of its route load, $\eta_i = q_i / Q_{r(i)}$. These dynamic signals indicate whether a route is nearly saturated and how much a given node contributes to that saturation.

Search meta-features (2). The current SA temperature, normalized to $[0, 1]$ over the schedule range, and the fraction of the search budget remaining. They are identical across nodes of the same state and let the policy modulate its behavior along the annealing schedule (e.g., bolder restructuring at high temperature, fine repairs near the end).

Group	Features	Dim
Static	$\mathbf{p}_i, \mathbf{1}_{\{i=0\}}, \theta_i, d_{i0}, q_i / Q$	6
Topology	$\mathbf{p}_{\text{prev}(i)}, \mathbf{p}_{\text{succ}(i)}$	4
Neighbor ranks	rank of $\text{prev}(i), \text{succ}(i)$	2
Relative demand	$q_i / \max_j q_j$	1
Spatial density	$\mu_5, \mu_{N/10}, \mu_{N/3}$	3
Local quality	$\Delta_i, d_{i,\text{cent}}$	2
Capacity status	$L_{r(i)}, S_{r(i)}, \eta_i$	3
Meta	temperature, remaining budget	2
Total		23

Table 4: Summary of the node feature vector $\mathbf{x}_i$.

Conditional pair features (stage 2, optional). In addition to the per-node vectors above, the second stage of the policy can receive descriptors of the candidate move itself, computed only for the selected first node u and every candidate partner v : the exact insertion cost $d(v, u) + d(u, \text{succ}(v)) - d(v, \text{succ}(v))$ together with the net cost change of the full move (insertion cost minus the removal saving of u), and the normalized distance-ranks of the edges the move would create. Because they are evaluated for a single u , these descriptors preserve the $\mathcal{O}(N)$ per-step complexity.

Appendix B: Operator–Feasibility Study, Full Results

This appendix collects the complete diagnostics of the operator $\times$ feasibility sweep summarized in Table 2. In the plot legends, the internal configuration names map to the paper terminology as `valid` = proactive masking, `free` = indirect representation (greedy split), `rm_depot` = reactive rejection.

Operator definitions. Figure 5 illustrates the three pairwise operators on two route fragments. The same action $a = (i, j)$ triggers a qualitatively different transformation under each operator, which is why a policy is trained for one fixed operator and learns a proposal distribution tailored to it.

Feasibility strategies. Under capacity constraints, many actions produce an infeasible neighbor $H(s, a) \notin \mathcal{F}$ — the central obstacle in extending neural SA from unconstrained problems to the CVRP. Three ways of keeping the search inside $\mathcal{F}$ are compared, one policy per operator–strategy cell (3 seeds per cell, 27 runs):

- *Proactive filtering (action masking).* The policy is restricted to the feasible subset $\mathcal{A}_f(s) = \{a : H(s, a) \in \mathcal{F}\}$ by masking infeasible actions before sampling. The conditional factorization makes this cheap: the first node i is selected freely, and only then are the feasible partners $\{j : H(s, (i, j)) \in \mathcal{F}\}$ computed and used to mask the second stage — $\mathcal{O}(N)$ checks per step instead of $\mathcal{O}(N^2)$. Every proposal is feasible by construction; since the fleet is unlimited, a node can always open a new route, so the feasible set is never empty.
- *Reactive rejection.* The policy samples any action from its learned distribution and the resulting neighbor is checked

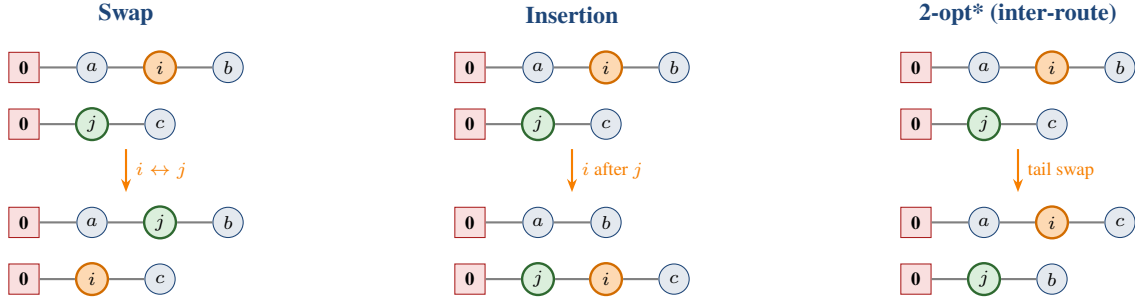


Figure 5: The three pairwise operators on two route fragments (top: before, bottom: after). *Swap* exchanges i (orange) and j (green). *Insertion* removes i and reinserts it immediately after j . *2-opt** exchanges the route suffixes following i and j ; when i and j belong to the same route, the operator instead reverses the path segment between them (standard 2-opt).

afterwards: if $s' \notin \mathcal{F}$, the move is rejected and the current solution retained. This avoids the filtering overhead but can waste both compute and learning signal when many sampled moves are infeasible.

- *Indirect representation.* Feasibility is delegated to the representation: the solution is encoded as an unconstrained permutation of the customers, the policy modifies the permutation, and a deterministic *greedy split* heuristic packs it back into capacity-feasible routes (Vidal 2016). Every state is feasible by construction and the policy never handles constraints, but the reconstruction runs at every step and its cost grows with the problem size.

Main effects. Table 5 averages each factor level over the other factor. Insertion and masking win marginally as well as jointly, so the winning cell of Table 2 is not an interaction artefact: the best operator and the best feasibility strategy are individually best as well. Relocating a single node is the move that best matches a learned pointer to a promising node pair, whereas the value of a swap or a segment reversal depends more delicately on both endpoints simultaneously, making the credit assignment noisier — visible in the swap column’s large seed variance. Reactive rejection is uniformly the worst strategy for every operator: discarding infeasible proposals wastes both search budget and learning signal, and the policy must spend capacity learning the feasibility boundary instead of move quality. The indirect (greedy-split) representation is competitive in cost for insertion but pays the reconstruction at every step. The margin of the winning cell over the runner-up (insertion + indirect, +0.29) is an order of magnitude above the insertion-row seed stds (0.03–0.04), and the runner-up shares the operator, so the operator choice stands even if the two leading feasibility strategies were tied.

Compute. Table 6 reports the evaluation wall-clock of every cell. Quality and compute are essentially uncorrelated over the grid — the cheapest column (rejection) is the worst on quality, and the most expensive cell (2-opt(*)) under masking) still loses to insertion — so the quality ranking of the main text is not a compute artefact.

Per-cell diagnostics. Figures 6–8 give three complementary views of the grid: the cost heatmap, the cost–compute trade-off, and the per-seed spread of the four best cells. Seed

Operator	Cost	Feasibility	Cost
Insertion	20.00	Masking	18.37
2-opt(*)	22.02	Indirect	20.36
Swap	26.03	Rejection	29.31

Table 5: Main effects: mean final cost of each factor level averaged over the other factor. Insertion and masking win marginally as well as jointly.

Operator	Masking	Indirect	Rejection
Insertion	173	172	152
Swap	170	167	148
2-opt(*)	301	177	157

Table 6: Mean wall-clock time (s) to evaluate the full 10,000-instance test set for 10^4 SA steps. Rejection is cheapest because infeasible proposals are simply discarded; 2-opt(*) under masking pays $1.7\times$ the compute of every other cell for its suffix-exchange feasibility checks.

variance grows quickly as cell quality degrades, so the winning cell is also the most reproducible one.

Appendix C: Feature Ablation, Full Results

This appendix collects the protocol, complete plots, and secondary tables of the feature study behind the “Node representation” paragraph and Table 3, in the order of the main-text narrative: single-group diagnostics, thematic (cluster) removals, the head-to-head validation of the candidate best sets, the targeted single-group refinements, and the conditional pair descriptors. In the plot labels, the internal group names map to the paper terminology of Table 4 as $\text{neighbor_rank} = \text{neighbor distance-ranks}$, $\text{demand_max} = \text{relative demand}$, $\text{density}_{5/10\%/33\%} = \mu_5/\mu_{N/10}/\mu_{N/3}$, $\text{route_pct} = \text{load ratio } L_{r(i)}$, $\text{slack} = \text{slack } S_{r(i)}$, and $\text{node_pct} = \text{load share } \eta_i$.

Protocol and reference points. The node representation is validated with three complementary protocols, all trained on the winning operator–feasibility pairing. *Leave-one-out* removes a single group from the full set and measures the

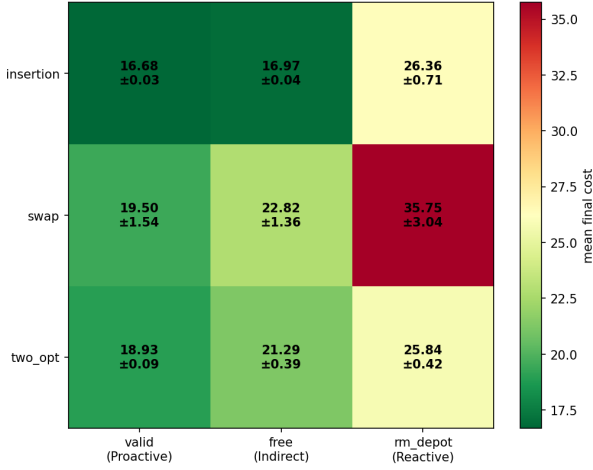


Figure 6: Mean final cost over the operator $\times$ feasibility grid (green = better). The insertion/masking corner is the optimum; the rejection column is uniformly poor across all operators.

degradation; *thematic* ablations remove a whole correlated cluster at once (e.g., all three density scales, or all capacity signals), exposing groups whose members are individually redundant but collectively useful; *isolation* adds a single group to a minimal base containing only the static and meta features, measuring its standalone value. From these measurements, candidate minimal feature sets are constructed — keeping groups whose removal hurts, dropping those that fail both the leave-one-out and the isolation test, and keeping one representative per correlated cluster — and confirmed seed-paired against the full set at 5 seeds; the conditional pair descriptors of the second stage are evaluated last, on top of the chosen set. The full 13-group representation reaches 16.678 ± 0.026 , against 20.084 ± 0.049 for the static+meta base: the solution-dependent features cut cost by 3.41 routing units ($\approx 17\%$). The pooled 3-seed floor of the diagnostic sweep is $\sigma \approx 0.18$, inflated by the single high-variance no-static configuration (typical seed std ≈ 0.05).

Single-group diagnostics. Table 7 reports the leave-one-out and isolation measurements. Four groups clear the noise floor at the margin: static (whose removal is catastrophic, +3.84), detour, topology, and meta. In isolation, the geometric and relational signals carry almost all the standalone value — detour (+2.60), topology (+2.19), neighbor ranks (+1.26), centroid (+0.48) — while the three capacity signals *hurt* when added alone to the base, and no density scale helps on its own. Applied mechanically, the single-feature pruning rule would therefore retain only six groups (static, meta, topology, neighbor ranks, detour, centroid) and drop the density, capacity, and relative-demand groups — the rule that the best-set validation below overturns. Figure 9 ranks all 30 ablation configurations and makes the two-cluster structure visible: leave-one-out runs stay near the full-set reference while isolation runs stay near the base, i.e., no single group is responsible for the 17% gap between the two. Figure 10

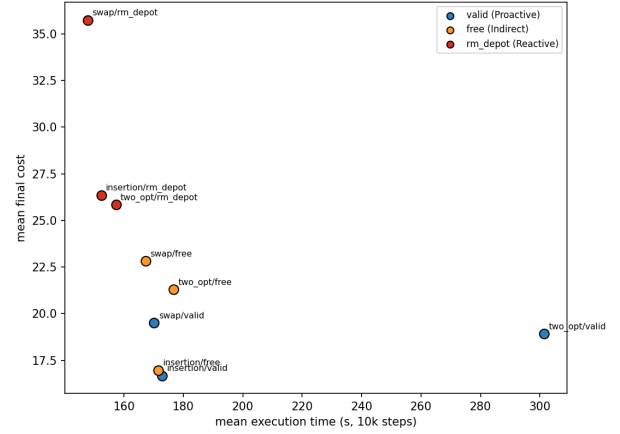


Figure 7: Cost against evaluation time for the nine cells. Insertion/masking sits at the bottom left (best on both axes); 2-opt(*)/masking spends $1.7\times$ the compute and remains worse.

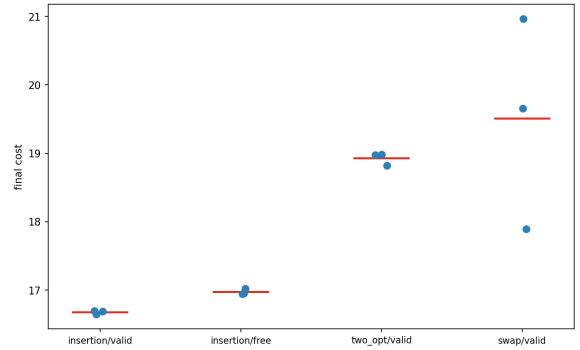


Figure 8: Per-seed final cost for the four best cells. The winning cell is tightly clustered; seed variance grows quickly as quality drops.

shows the per-seed spread of the reference configurations and explains why the pooled 3-seed floor is inflated by the single high-variance no-static run.

Thematic ablation. Table 8 and Figure 11 report the removal of whole correlated clusters. This is the protocol that exposes collective value invisible to leave-one-out: no single capacity feature clears the floor on its own, but removing all three costs +0.343.

Best-set validation. The candidate sets of Table 3 were retrained head-to-head at 5 seeds on a batch with a clean floor ($\sigma_{\text{floor}} \approx 0.057$; per-set seed std between 0.04 and 0.07). Seed-paired against the full set, the pruned sets lose $+0.337 \pm 0.091$ (set7), $+0.405 \pm 0.077$ (nodepct6), $+0.417 \pm 0.049$ (centroid6), and $+0.479 \pm 0.075$ (core5) — every candidate significantly worse, degrading monotonically as groups are removed — while dropping the load ratio alone is a statistical tie (-0.035 ± 0.106). Figure 12 gives the per-seed view of the confirmation: the left panel shows that the 12-group and full-set clusters overlap and sit cleanly below every pruned candidate, and the right panel shows the seed-paired deltas

Group	Δ_{LOO}	Δ_{iso}
Static	+3.84 ± 0.85	(in base)
Detour Δ_i	+0.60 ± 0.08	+2.60 ± 0.07
Topology	+0.46 ± 0.08	+2.19 ± 0.15
Meta	+0.31 ± 0.08	(in base)
Slack $S_{r(i)}$	+0.12 ± 0.04	−0.42 ± 0.31
Centroid $d_{i,\text{cent}}$	+0.10 ± 0.08	+0.48 ± 0.06
Density $\mu_{N/3}$	+0.09 ± 0.12	−0.08 ± 0.06
Neighbor ranks	+0.03 ± 0.05	+1.26 ± 0.09
Relative demand	+0.03 ± 0.09	−0.01 ± 0.02
Density μ_5	+0.01 ± 0.01	−0.02 ± 0.09
Load share η_i	+0.01 ± 0.04	−0.42 ± 0.10
Density $\mu_{N/10}$	−0.03 ± 0.07	−0.09 ± 0.09
Load ratio $L_{r(i)}$	−0.06 ± 0.08	−0.44 ± 0.36

Table 7: Single-group signals, sorted by marginal value. Δ_{LOO} : cost increase when the group is removed from the full set (positive = useful at the margin). Δ_{iso} : cost reduction when the group alone is added to the static+meta base (positive = useful standalone). Bold: exceeds the noise floor. Static and meta belong to the isolation base and have no standalone entry.

Theme removed	Cost	Δ vs. full
Relational (topology + nbr. ranks)	17.277	+0.599 ± 0.016
Geometry (detour + centroid)	17.274	+0.596 ± 0.033
Capacity (L , S , η)	17.021	+0.343 ± 0.059
Density (μ_5 , $\mu_{N/10}$, $\mu_{N/3}$)	16.822	+0.145 ± 0.033

Table 8: Thematic ablation (3 seeds, seed-paired vs. the full set; positive Δ = the theme helps). The relational and geometry clusters are the two most valuable; capacity clears the typical seed noise while density does not.

against the noise floor — the evidence behind the reversal of the single-group pruning rule.

Targeted refinements. Table 9 and Figure 13 isolate the two borderline groups. Dropping the load ratio alone is a tie with the full set (reproduced independently at 3 and 5 seeds); additionally dropping the neighbor ranks costs an incremental $+0.200 \pm 0.066$, several times the floor — the neighbor ranks, not the load ratio, are the group that must stay.

Conditional pair descriptors. On top of the retained set, the two second-stage conditioning signals (edge distance-ranks and exact insertion cost, defined in Appendix A) were swept standalone and as a 2×2 cross-product (3 seeds, noise floor $\sigma \approx 0.071$). Both *hurt*: enabling the distance-rank descriptor costs $+0.14 \pm 0.11$ and the insertion-cost descriptor $+0.18 \pm 0.07$, each adding ≈ 20 s of wall-clock; the 2×2 main effects match the standalone deltas ($+0.13$ and $+0.17$) and the interaction is negligible (-0.02 , within the floor), so the effects are additive and the both-on cell is the worst (Figure 14). The extra signal apparently distracts more than it informs: the per-node features already encode the local geometry the descriptors summarize.

Set	Groups	Cost	Time (s)	Δ vs. full
full set	13	16.678	179	ref
− load ratio	12	16.619	166	-0.059 ± 0.076
− load ratio, − ranks	11	16.819	154	$+0.141 \pm 0.022$

Table 9: Targeted single-group refinements (3 seeds, seed-paired; mini-batch noise floor ≈ 0.035 ; per-set seed std between 0.02 and 0.05). Dropping the load ratio alone is a statistical tie with the full set; additionally dropping the neighbor ranks costs an incremental $+0.200 \pm 0.066$. The apparent 7% speedup of the 12-group set did not replicate at 5 seeds (Table 3) and is machine-load noise.

Critic	Cost
multi-statistic	16.635 ± 0.038
feed-forward	16.693 ± 0.006

Table 10: Critic comparison (3 runs per arm; noise floor ≈ 0.027). The multi-statistic critic improves the mean final cost by 0.058, about twice the floor. The critic is used only during training, so deployment cost is unchanged.

Appendix D: Architecture Study, Full Results

This appendix collects the candidate definitions, complete grids, and per-seed diagnostics of the architecture study behind the “Policy and learning” paragraph of the main text, one paragraph per axis in the order they were swept (critic, capacity, global context, shared encoder, distribution shaping), each sweep inheriting the previous winner. In the plot labels, the internal names map to the paper terminology as `deepsets` = multi-statistic critic, `ff` = mean-pooled feed-forward critic, `seq` = the incumbent actor with independent per-stage scorers, `shared` = the shared-encoder variant, `gc` = pooled global context, `bil` = bilinear second-stage head (0 = concatenation), and networks are written width/depth, e.g. `64/2L`.

Critic. The baseline critic scores each node with a small MLP and averages the scalar outputs into the value estimate, which makes V_ϕ a linear functional of the mean node representation. The alternative is the multi-statistic variant of the training section: nodes are encoded into embeddings, pooled with mean, max, and standard deviation, and mapped to the value by a nonlinear head — so dispersion information (clustered against scattered instances) survives the pooling. Table 10 compares the two. The critic only shapes the advantage estimates during training and is discarded afterwards, so the comparison is on solution quality alone.

Actor capacity. The embedding width and the number of hidden layers of the two scoring networks are swept jointly, trading representational capacity against per-step inference cost. Table 11 gives the full width $\times$ depth grid: the full spread from the smallest to the largest network is 0.079, no cell clears the floor (≈ 0.053) against the 32×1 incumbent, and wall-clock rises by up to 38%; the faint monotone trend favors width over depth (main-effect spans 0.069 against

Net	Cost	Time (s)	Δ vs. 32×1
64×2	16.589 ± 0.004	238	-0.049 ± 0.039
64×1	16.591 ± 0.084	207	-0.047 ± 0.060
32×2	16.601 ± 0.008	187	-0.037 ± 0.030
32×1	16.637 ± 0.036	173	ref
16×2	16.649 ± 0.083	165	$+0.011 \pm 0.093$
16×1	16.668 ± 0.042	160	$+0.031 \pm 0.063$

Table 11: Actor capacity grid (embedding width $\times$ hidden layers), sorted by mean cost; Δ is seed-paired against the 32×1 incumbent. No cell clears the noise floor (≈ 0.053); the width main effect spans 0.069 against 0.019 for depth.

0.019). Figure 15 adds the seed-paired deltas against the noise band — every cell falls inside it — and Figure 16 shows the cost–compute view: quality improves only marginally along a steeply rising compute axis, the signature of a capacity plateau.

Global context. The *global context* option pools the node representations into a summary g (mean, max, and standard deviation) appended to the input of both stages — the cheapest way to give the per-node scores access to instance-level information, at one pooling operation per step. It backfires: the mean cost rises from 16.633 to 17.143 (seed-paired $+0.508 \pm 0.496$, one seed degrading by $+1.08$) and the evaluation wall-clock from 180 to 255 s ($+48\%$) — the instance-level summary distracts the per-node scores rather than informing them. Figure 17 shows the per-seed effect: every seed degrades, one dramatically, so the regression is not a variance artefact.

Shared encoder. Figure 18 illustrates the variant: each node is encoded once into an embedding $\mathbf{h}_i$, and both selection stages are lightweight heads over the shared embeddings, the conditional score being computed either by a concatenation head over $[\mathbf{h}_j | \mathbf{h}_i]$ or as a bilinear compatibility $\mathbf{q}^\top \mathbf{k} / \sqrt{d}$ between a query built from the selected node and a key per candidate. Table 12 and Figure 19 report its 2×2 grid (head type $\times$ context) against the incumbent: every cell sits $+3.20$ to $+3.53$ above the reference — a 19% quality loss, more than $18\times$ that batch’s noise floor — so forcing both stages through one trunk destroys the pair-scoring ability at this problem size; the failure is structural rather than a bad corner of the grid.

Distribution shaping. Two flags tame the proposal distribution without adding capacity: *logit clipping*, which bounds the logits to $C \tanh(z)$ before the softmax so the distribution cannot collapse to a near-deterministic choice (Kool, van Hoof, and Welling 2019), and a *learnable softmax temperature* per stage. Table 13 and Figure 20 report their 2×2 grid: only clipping clears the floor (all three seeds improve), while the learnable temperature is within noise on its own (-0.050 ± 0.069) and slightly degrades the clipped configuration when stacked, with doubled seed variance — one more trainable knob for PPO to destabilize.

Stage-2 head	Context	Cost	Time (s)	Δ vs. ref
bilinear	on	19.830	196	$+3.196 \pm 0.103$
bilinear	off	19.860	168	$+3.226 \pm 0.163$
concat	off	20.154	192	$+3.520 \pm 0.251$
concat	on	20.160	236	$+3.526 \pm 0.099$

Table 12: Shared-encoder grid, sorted by mean cost; Δ is seed-paired against the incumbent actor re-run with context off (16.633 ± 0.036 , 180 s). Per-cell seed std between 0.06 and 0.26; noise floor ≈ 0.176 . Within the shared family the bilinear head beats concatenation (main effect -0.31) and the context toggle is neutral (-0.01), but every cell is ≈ 3 units worse than the reference.

Clipping	Temp.	Cost	Time (s)	Δ vs. ref
$C = 10$	fixed	16.457	177	-0.173 ± 0.059
$C = 10$	learned	16.481	179	-0.149 ± 0.085
off	learned	16.580	176	-0.050 ± 0.069
off	fixed	16.630	175	ref

Table 13: Distribution-shaping grid, sorted by mean cost; Δ is seed-paired against the both-off incumbent. Per-cell seed std between 0.04 and 0.11; noise floor ≈ 0.072 . Only logit clipping clears the floor; the learnable temperature does not help it (clipping main effect -0.136 , temperature -0.014).

Appendix E: Search-Loop Components, Full Results

This appendix collects the full rankings and per-seed diagnostics behind the reward, annealing, acceptance, and initialization studies of the main text, one paragraph per axis. All four batches follow the standard protocol (3 seeds, 10,000 SA steps at half precision on the Nazari $N = 100$ test set); the incumbent configuration reproduces at 16.459–16.460 across the four batches, matching the adopted cell of the architecture study (Table 13).

Reward. The reward decides which behavior PPO reinforces: per-step improvement, new records, or only the final outcome. Because the SA loop already owns part of the exploration — worsening moves are accepted by the Metropolis criterion, not chosen by the policy — it is not obvious how much of the trajectory should be credited to the actor. Let Δ_t denote the cost improvement realized by the executed transition (zero when the move is rejected) and B_t the best cost found so far. The compared variants are: *immediate*, $r_t = \Delta_t$ with a fixed penalty for infeasible proposals; *acceptance-aligned*, which rewards accepted improving moves by Δ_t but deliberately assigns zero to accepted worsening moves (exploration is SA’s responsibility, not the policy’s fault) and a small constant penalty to rejected proposals; *best-so-far*, $r_t = \max(0, B_{t-1} - B_t)$, which only rewards new records; *terminal*, a single end-of-episode reward equal to the relative improvement over the initial cost; *mixtures*, either a fixed convex combination of the immediate and best-so-far signals or a scheduled interpolation that starts from the dense acceptance-aligned signal early

Reward	Cost
immediate (incumbent)	16.459
fixed mixture	16.460
sched. $\rightarrow$ best-so-far	16.503
best-so-far	16.510
acceptance-aligned	16.516
sched. $\rightarrow$ terminal	16.531
step-penalty (const.)	17.198
step-penalty (init)	17.549
terminal	19.411

Table 14: Reward ablation: mean final cost over 3 seeds, sorted; the rule separates the dense stable cluster from the sparse and penalty-based rewards. Wall-clock is flat across arms (173–177 s). Per-seed diagnostics in Figure 21.

in training and shifts toward the sparser best-so-far or terminal signals; and *step-penalty*, a negative per-step reward proportional to the current best cost, pressuring fast descent. In the plot labels, the internal reward names map to this terminology as *immediate* = immediate, *sa_aligned* = acceptance-aligned, *global_best* = best-so-far, *hybrid* = the fixed mixture ($\alpha = 0.5$), *curriculum* and *curriculum_terminal* = the scheduled interpolations toward best-so-far and terminal respectively, and *step_penalty_init* and *step_penalty_16* = the per-step penalty normalized by the initial cost and by a fixed constant close to the typical final cost.

Table 14 splits the nine candidates into two regimes (noise floor 0.042, the pooled seed std of the stable cluster). Every reward built on dense per-step improvement lands in a band only 0.07 wide: the incumbent immediate reward is best at 16.459, the fixed mixture is a statistical tie (seed-paired $+0.001 \pm 0.042$), and the acceptance-aligned, best-so-far, and scheduled variants trail by at most 0.07 — the actor is largely insensitive to the exact shaping as long as the signal is dense. Removing the per-step credit is catastrophic: the terminal reward loses 2.95 routing units with seed swings of almost a full unit, and the step-penalty schemes lose 0.7 to 1.1 with similarly unstable training — the penalty fights the improvement signal instead of complementing it. Figure 21 zooms on the stable cluster and shows the seed-paired deltas against the noise band.

Cooling schedule. The cooling schedule controls how quickly the acceptance criterion hardens from tolerant exploration to strict descent; a learned proposal could in principle prefer a different cooling profile than blind SA. The *shape* comparison holds the temperature range (T_0, T_{final}) fixed and opposes the *geometric* decay $T_{k+1} = \alpha T_k$ to a piecewise-constant *step* schedule that drops the temperature at one half, two thirds, and three quarters of the horizon. It is a statistical tie: 16.460 against 16.473, a seed-paired -0.013 ± 0.034 against a noise floor of 0.044, with the per-seed sign split and identical wall-clock (Figure 22). The *range*, by contrast, holds the largest single-knob improvement of the whole validation: a 3×2 grid retrains the policy over $T_0 \in \{0.5, 1, 2\}$ and $T_{\text{final}} \in \{0.01, 0.1\}$ (Table 15) and

$T_0 \backslash T_{\text{final}}$	0.01	0.1
0.5	16.342	16.475
1	16.343	16.508
2	16.354	16.486

Table 15: Temperature-range grid: mean final cost over 3 seeds; rows = initial temperature T_0 , columns = final temperature T_{final} (incumbent in bold).

Train \ Test	Metropolis	Accept-all
Metropolis (incumbent)	16.459	16.578
Accept-all	16.432	16.526

Table 16: Metropolis 2×2 cross: mean final cost over 3 seeds; rows = training setting, columns = deployment setting. Per-seed effects in Figure 23.

splits cleanly by final temperature — every $T_{\text{final}} = 0.01$ cell beats every $T_{\text{final}} = 0.1$ cell, and the incumbent (1, 0.1) is in fact the worst cell of its own grid. Cooling deeper gains -0.165 ± 0.034 at the incumbent T_0 , four times the 0.041 floor, with a unanimous sign at every starting temperature; the initial temperature is inert (at $T_{\text{final}} = 0.01$ the three starting points are a three-way tie). Since 0.01 is the boundary of the tested grid, an even lower floor cannot be ruled out.

Metropolis acceptance. Once the proposal is learned, does the stochastic acceptance still earn its keep — and should the policy be trained with it in the loop? A 2×2 ablation enables or disables the Metropolis criterion independently at training and at test time (the “off” arm accepts every proposed move), quantifying whether the learned proposal replaces the stochastic acceptance or complements it. The cross (Table 16) shows the criterion earns its keep at deployment, not at training: removing the acceptance at test time costs $+0.119 \pm 0.025$ over the incumbent-trained actor and $+0.094 \pm 0.009$ over the greedy-trained one — several times the 0.034 floor and unanimous across seeds — whereas the training-time flag is within noise at the standard test-on deployment ($+0.027 \pm 0.045$, sign split). Only when deploying greedily does greedy training pay ($+0.052 \pm 0.050$), a mild train/test-consistency effect outside the regime we deploy in. Figure 23 decomposes the cross into its two effects; the greedy-trained arms also show markedly lower seed variance (0.010/0.020 against 0.040/0.050).

Initialization. Every episode starts from a constructive solution, both during training and at deployment, and the two roles need not agree: at training time the construction fixes the state distribution the policy learns from, while at deployment it is a free knob of the solver. Training starts from a single constructive heuristic — random assignment, nearest neighbor, sweep, or single-customer routes — or from a progressively introduced mixture of all four; every trained policy is additionally cross-tested from all five test-time constructions (the four above plus Clarke–Wright savings). The heuristics appear under their code names *random* (random assignment),

Train \ Test	Rand.	Sweep	NN	C-W	Isol.
random (incumbent)	16.460	16.467	16.467	16.307	16.444
sweep	17.548	16.950	16.854	16.271	17.148
nearest neighbor	19.228	17.729	17.401	16.381	18.837
single-customer	17.736	17.725	17.753	16.442	17.706
mixed	16.682	16.663	16.700	16.357	16.673

Table 17: Initialization cross: mean final cost over 3 seeds, rows = training construction, columns = test-time construction (NN = nearest neighbor, C-W = Clarke-Wright savings, Isol. = single-customer routes). The random-trained row is uniformly the best, and its spread across test constructions is only 0.160 although the construction costs span 16.4 (C-W) to 104.0 (single-customer). The structured-training rows recover only in the C-W column, where the construction itself already sits at the fixed point of the search — that column measures the deployment, not the actor. Worst case over deployments: 16.467 (random), 16.700 (mixed), 17.5–19.2 (structured).

sweep, nearest_neighbor, Clark_and_Wright (Clarke-Wright savings), and isolate (single-customer routes); their construction costs on the test set are 57.9, 30.5, 19.0, 16.4, and 104.0 respectively. Table 17 gives the full train $\times$ test cross. At deployment the construction barely matters — the random-trained actor lands in a band only 0.160 wide, the one real edge being a Clarke-Wright start, which already sits at the fixed point of the search (-0.153 ± 0.021 against the 0.063 stable floor). At training time it is decisive: actors trained from the structured heuristics lose 1.1 to 2.8 routing units at the standard deployment, unanimously and far past even the global 0.395 floor, and mixing all four constructions into training dilutes coverage, a small but unanimous regression ($+0.222 \pm 0.084$). Figure 24 shows the cross as a heatmap, and Figure 25 shows the two slices that support the main-text reading: training construction matters, deployment construction barely does, and the matched train/test diagonal — worse than the random-trained actor everywhere — rules out a mismatch explanation.

Appendix F: Optimization Hyperparameters, Full Results

This appendix documents the optimization-hyperparameter sweeps and their combined-configuration validation, in particular the reproducible divergence that fixed the final critic learning rate. Every axis before this one compared design alternatives; the plain numerical knobs of the optimizer were inherited, not tuned. With the architecture and search loop frozen, the knobs are swept around the incumbent one axis at a time: the actor and critic learning rates (a 3×3 grid), the entropy coefficient, and the rollout length — the number of SA steps collected per PPO update, which sets both the horizon over which advantages are estimated and how far into the cooling schedule the collected experience reaches. Every incumbent lies strictly inside its grid, so each sweep is a proper local search around the default. All batches follow the standard protocol (3 seeds per configuration, 10,000 SA

Actor \ Critic	3×10^{-4}	10^{-3}	3×10^{-3}
3×10^{-4}	16.447	16.435	16.396
10^{-3}	16.509	16.460	16.466
3×10^{-3}	16.509	16.502	16.510

Table 18: Learning-rate grid: mean final cost over 3 seeds; rows = actor rate, columns = critic rate (incumbent in bold).

Entropy coefficient	Cost
0	16.587
0.01 (incumbent)	16.459
0.05	17.242
Rollout length	
50	16.904
100 (incumbent)	16.460
200	16.404

Table 19: Entropy-coefficient and rollout-length sweeps: mean final cost over 3 seeds.

steps at half precision on the Nazari $N = 100$ test set); the per-batch noise floors are 0.031 (learning rates), 0.046 (entropy), 0.041 (rollout), and 0.041 (temperature range), and the incumbent reproduces at 16.459–16.508 across the four batches.

Learning rates. The actor learning rate is the axis that matters, and its effect is monotone (Table 18): the three worst cells all share the highest actor rate, the three best all share the lowest, and each grid step upward costs roughly 0.05 on average over the critic axis. The critic rate is a weaker, second-order knob. The best cell — actor 3×10^{-4} , critic 3×10^{-3} — improves on the incumbent by -0.064 ± 0.051 , unanimous across seeds against the 0.031 floor; the combined-configuration runs below explain why the adopted configuration nevertheless keeps the critic rate at 10^{-3} .

Entropy and rollout. The entropy coefficient is a clean confirmation of the default (Table 19, top): 0.01 sits at the bottom of a U, with both neighbors unanimously worse — removing the bonus costs $+0.128 \pm 0.021$ (the policy under-explores and collapses onto a narrow move set), over-weighting it costs $+0.783 \pm 0.036$ (near-uniform proposals). The rollout length is asymmetric (bottom): halving it to 50 is a large unanimous regression ($+0.444 \pm 0.015$) — the collected experience never reaches the cool end of the schedule where the hard moves live — while doubling it to 200 is a marginal, sign-split gain (-0.056 ± 0.053) at twice the collection cost per update.

Combined-configuration runs. The three wins — the lower actor rate, the longer rollout, and the deeper cooling floor of the annealing study — were each measured in isolation, so the joint configuration is retrained from scratch to test additivity. With the best-cell critic rate of 3×10^{-3} , five of six seeds land at 16.216 ± 0.048 , matching the sum of the individual effects; reducing the critic rate to the incumbent 10^{-3} converges on every seed at 16.230 ± 0.039 ,

Seed	Critic 3×10^{-3}	Critic 10^{-3}
0	16.265	16.257
1	19.865 / 19.872	16.268
2	16.219	16.186
3	16.258	16.209
4	16.152	—
5	16.186	—
mean (converged)	16.216 ± 0.048	16.230 ± 0.039

Table 20: Per-seed final cost of the combined configuration (actor rate 3×10^{-4} , rollout 200, $T_{\text{final}} = 0.01$) under the two candidate critic learning rates. Seed 1 under the 3×10^{-3} critic was trained twice independently; both runs diverge into the same degenerate basin. The 10^{-3} critic converges on every seed.

a seed-paired -0.223 ± 0.016 against the incumbent, unanimous — the adopted configuration. Two caveats carry over to the main text: the two winning values (actor rate and cooling floor) both sit on the boundary of their grids, and the divergence below shows that stacking individually-validated winners can create interactions none of them exhibits alone. Table 20 lists every training run of the two combined configurations. The aggressive variant (critic rate 3×10^{-3} , the best single cell of the learning-rate grid) was extended from 3 to 6 seeds after the first divergence, and the failing seed was retrained a second time from scratch: both runs land at 19.86–19.87, so the failure is deterministic given the seed, not a flaky run. The same seed trains without incident under the incumbent configuration (16.473) and under the final configuration (16.268), so the seed itself is not pathological — the divergence is created only by the combination of knobs, each of which was stable across all its seeds in isolation.

Failure mechanism. The quantity that overflows is not the learning rate itself but the product of learning rate and advantage, and it is the fast critic that manufactures the oversized advantages. With a 200-step rollout cooling to $T_{\text{final}} = 0.01$, the long low-temperature tail of every collected episode is a regime where the Metropolis rule rejects nearly every non-improving move; a rejected move leaves the state unchanged and its immediate reward is exactly zero, so early in training — when the near-random policy finds few improving moves — the return signal is mostly zeros punctuated by rare spikes. A critic stepping $10\times$ faster than the policy it evaluates overshoots when fitting that target, and an occasional badly mis-scaled value estimate produces one outsized advantage, hence one outsized policy step; the updated policy then jumps far enough from the collection policy that the exponentiated log-ratio overflows in the backward pass and the run never recovers. The seed dependence is a race: a seed that lingers longer in the flat zero-reward regime gives the fast critic more opportunities to overshoot before the policy finds signal — the failing seed spent roughly 50 epochs there where a converging seed escaped in about 5. Matching the critic rate to the incumbent 10^{-3} keeps the advantages bounded and closes the race at no measurable cost: the two

variants are statistically indistinguishable on their common converged seeds (Table 20).

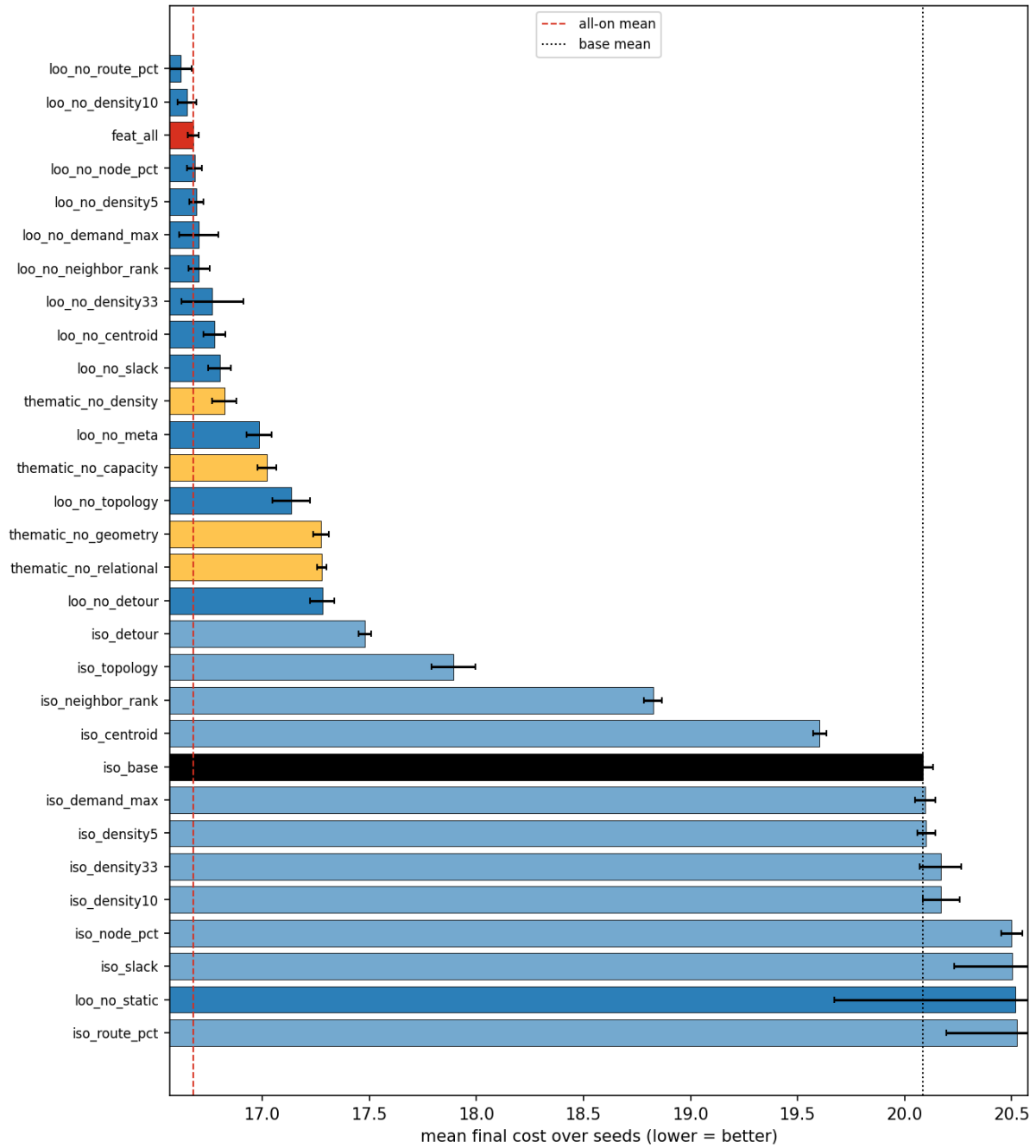


Figure 9: All 30 ablation configurations ranked by mean final cost. Leave-one-out configurations (dark blue) cluster tightly near the full-set reference (red dashed); isolation configurations (light blue) sit near the static+meta base (dotted).

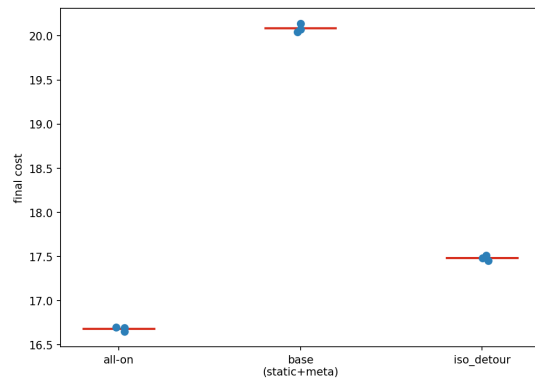


Figure 10: Per-seed final cost for the key reference configurations (red bars = means). Seed variance is tiny except for the no-static configuration, which inflates the pooled noise floor of the 3-seed sweep.

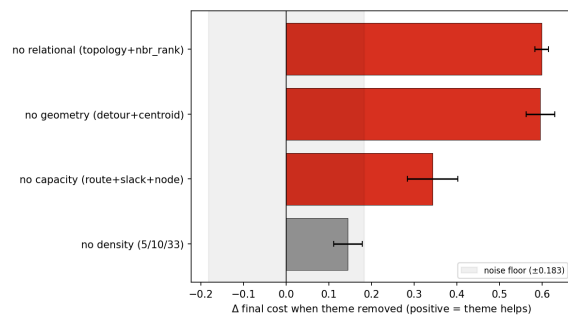


Figure 11: Thematic ablation (Table 8): removing the relational or geometry cluster costs ≈ 0.6 routing units each.

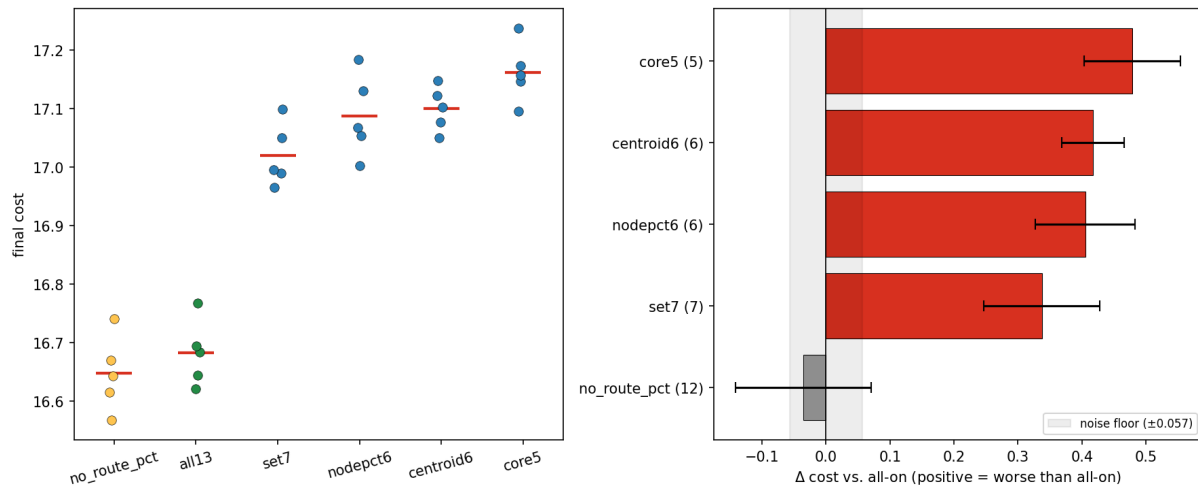


Figure 12: Best-set validation, per-seed view. Left: per-seed final cost — the 12-group and full-set clusters overlap and sit cleanly below every pruned set. Right: seed-paired Δ vs. the full set — the pruned sets (red) clear the noise floor by a wide margin; the 12-group set (green) sits below zero.

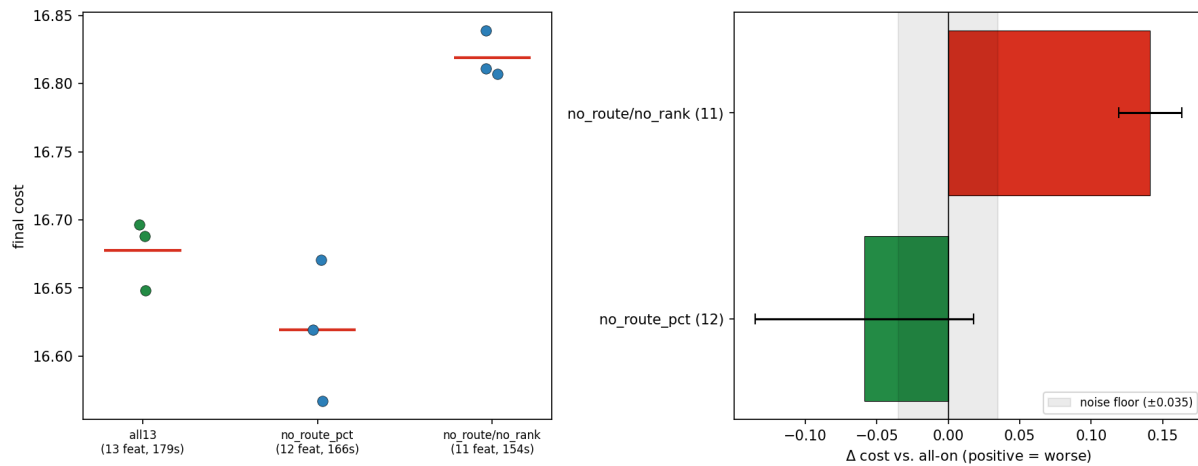


Figure 13: Targeted refinements (Table 9). Left: per-seed final cost — dropping the load ratio overlaps the full set, while additionally dropping the neighbor ranks lifts the whole cluster. Right: seed-paired Δ vs. the full set — the load-ratio removal (green) sits inside the noise floor, the neighbor-rank removal (red) clears it decisively.

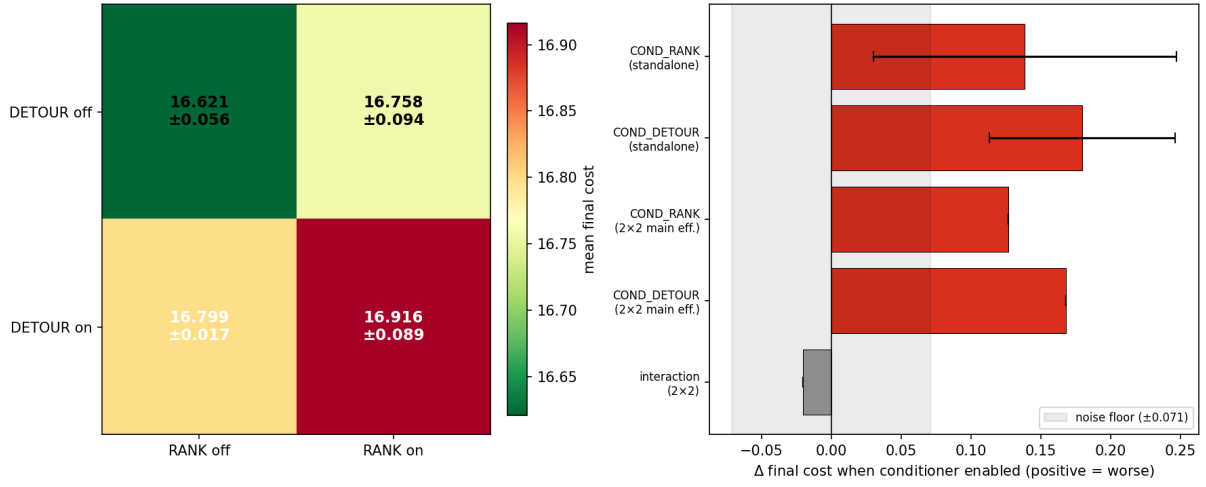


Figure 14: Conditional pair descriptors. Left: 2×2 grid of mean final cost — the both-off cell (plain retained set) is best, both-on is worst. Right: standalone deltas and 2×2 main effects — both descriptors sit clearly beyond the noise floor on the harmful side; the interaction is within it.

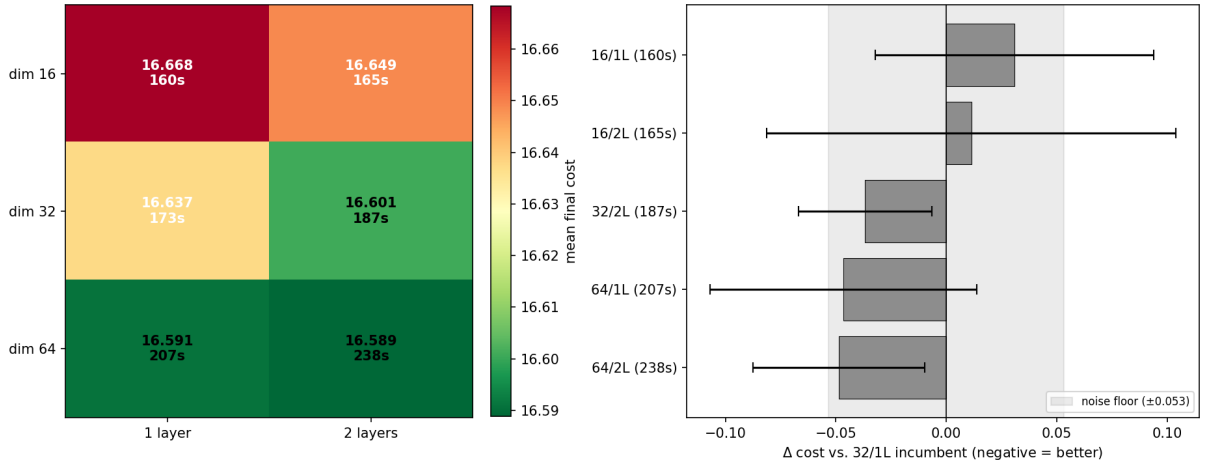


Figure 15: Actor capacity grid (Table 11). Left: mean final cost and wall-clock per cell (green = better). Right: seed-paired Δ against the 32×1 incumbent — every cell falls inside the shaded noise band, so no capacity change is a significant win.

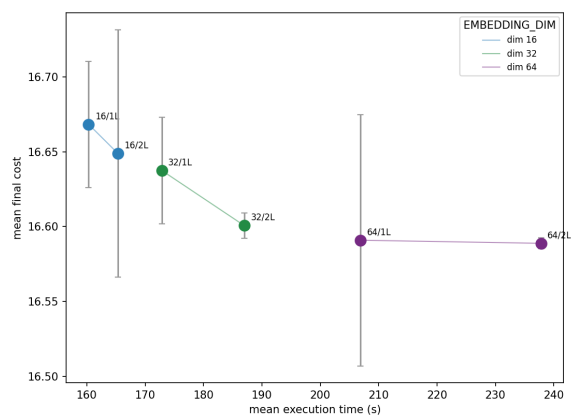


Figure 16: Cost against compute across the six capacity cells, colored by embedding width. Quality improves only marginally along a steeply rising compute axis — the hallmark of a capacity plateau.

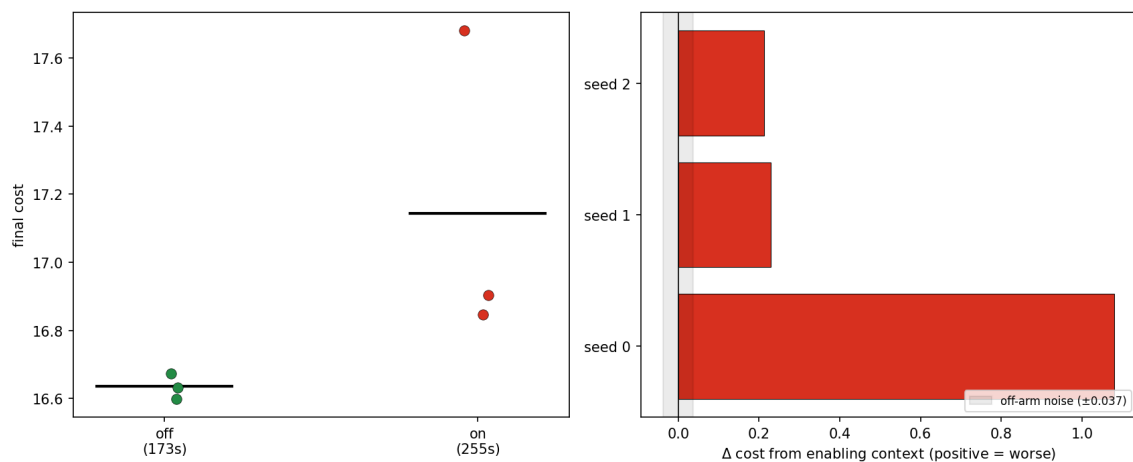


Figure 17: Global-context toggle. Left: per-seed final cost (black bars = means); the context-on arm is both worse and far more variable. Right: per-seed Δ from enabling context — every seed is positive (worse), one dramatically so, so the degradation is not a variance artefact.

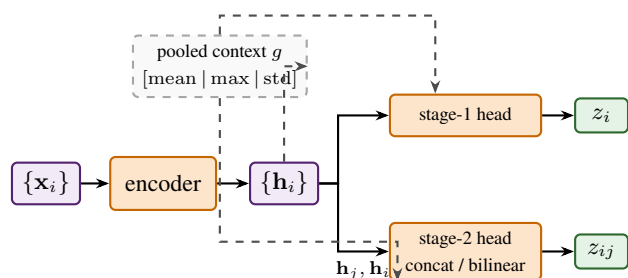


Figure 18: Shared-encoder actor variant. Each node is encoded once into an embedding h_i ; two lightweight heads then score the two stages over the shared embeddings, either by concatenation or as a bilinear compatibility between the selected node and each candidate. An optional pooled summary g of all node embeddings (dashed) is appended to both heads' inputs, giving every score access to instance-level context.

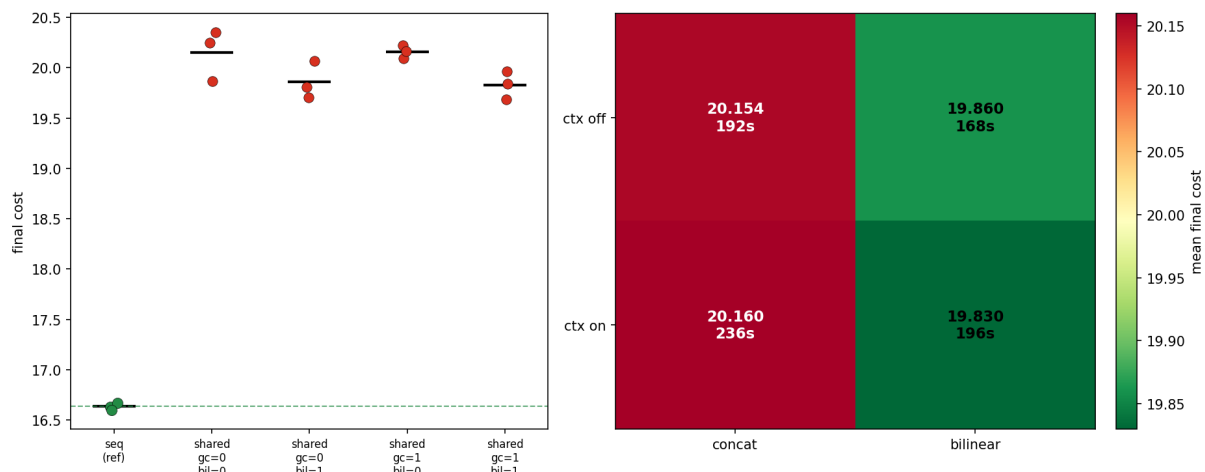


Figure 19: Shared encoder against independent scorers (Table 12). Left: per-seed final cost, incumbent reference (green) against the four shared cells (red); the dashed line marks the reference mean, and every shared seed sits far above it. Right: the shared 2×2 grid — the bilinear head beats concatenation and context is roughly neutral, but all four cells are ≈ 3 units worse than the reference.

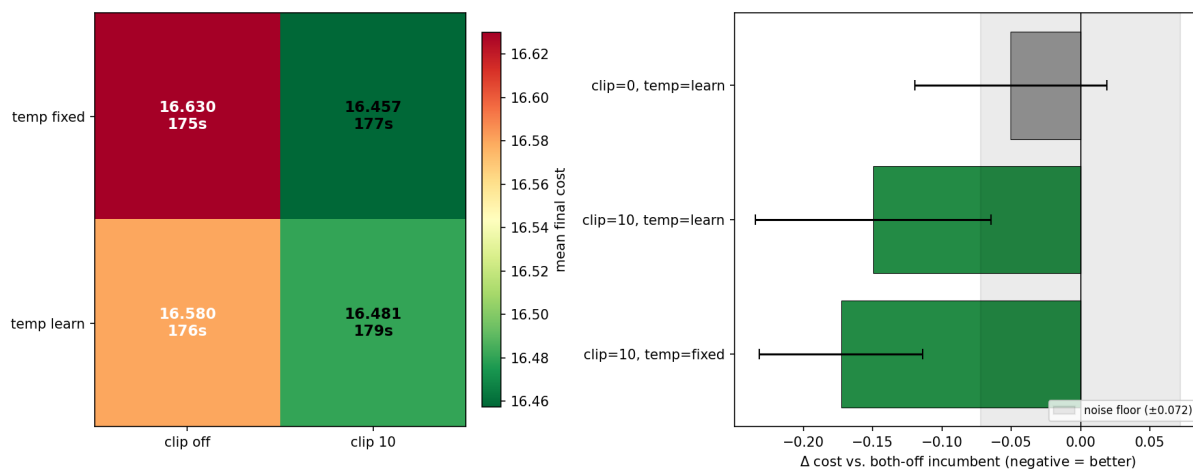


Figure 20: Distribution shaping (Table 13). Left: 2×2 grid of mean final cost and time (green = better); the clipping-on, fixed-temperature cell is the best. Right: seed-paired Δ against the both-off incumbent — only logit clipping escapes the noise band, and stacking the learnable temperature on top of it does not help.

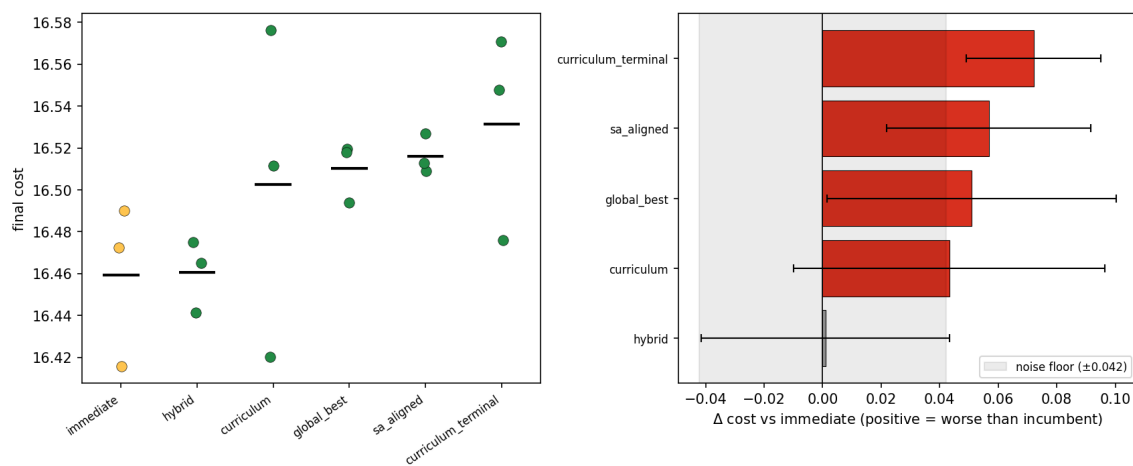


Figure 21: Zoom on the stable reward cluster. Left: per-seed final cost (black bars = means); the arms overlap heavily. Right: seed-paired Δ against the incumbent immediate reward — the fixed mixture sits inside the noise band (a tie), the remaining dense variants are marginally but consistently above zero.

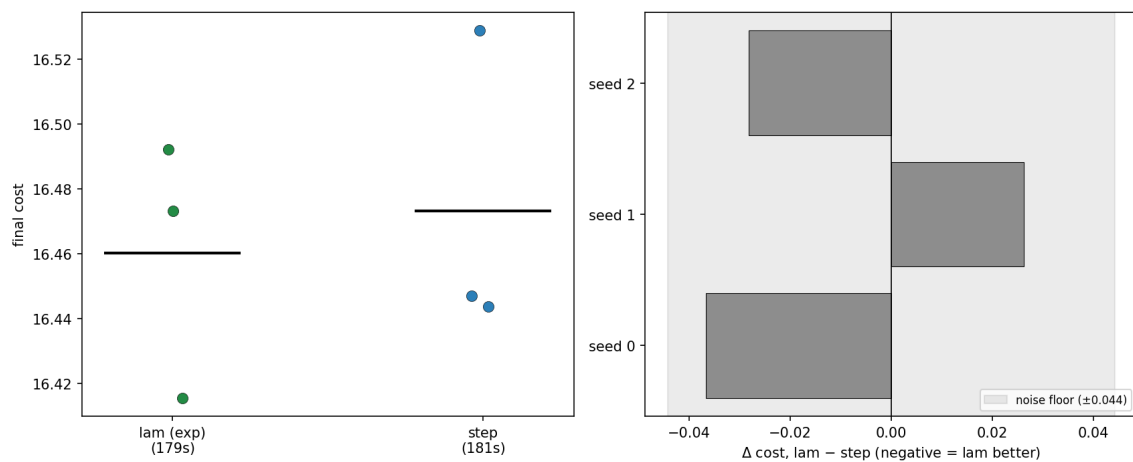


Figure 22: Cooling-schedule shape comparison. Left: per-seed final cost for the two schedules (black bars = means); the clusters overlap heavily. Right: per-seed paired Δ (geometric – step) — every bar lies inside the noise band and the sign is not consistent, the signature of a tie rather than an effect.

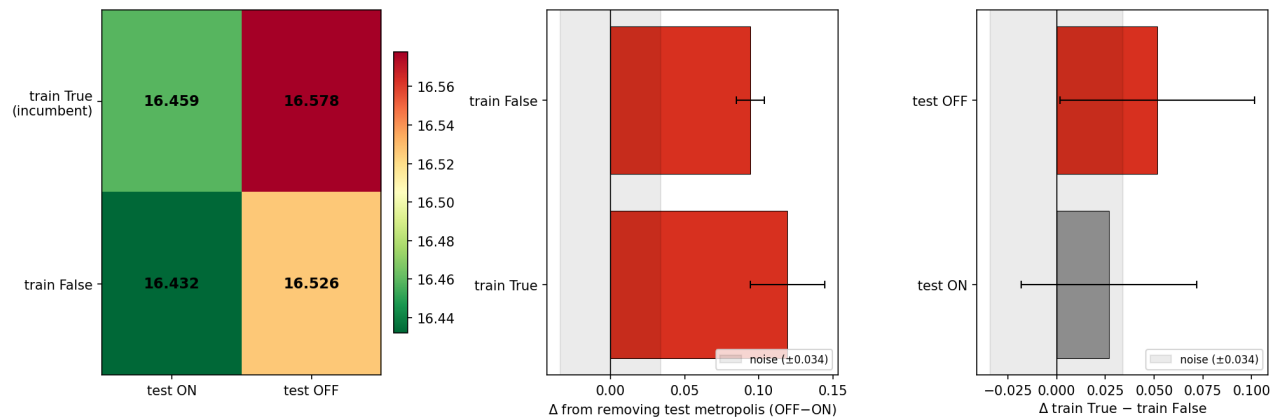


Figure 23: Metropolis train $\times$ test cross (Table 16). Left: 2×2 heatmap of mean final cost (greener = better). Center: cost of removing test-time Metropolis (off – on) at each training setting — both bars are well past the noise band, the dominant effect. Right: training-flag effect (on – off) at each deployment — the test-on bar sits inside the band (a tie), test-off just clears it.

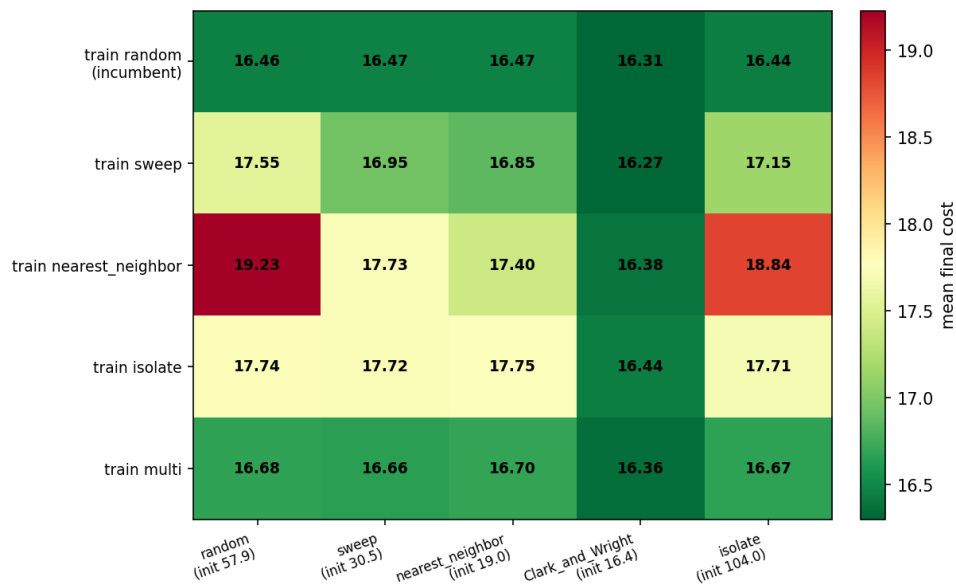


Figure 24: Initialization cross (Table 17): rows = training construction, columns = test-time construction, greener = lower final cost. The random-trained and mixed-trained rows are uniformly good; the three structured-training rows are poor and only recover in the Clarke–Wright column, where the construction itself already did the work.

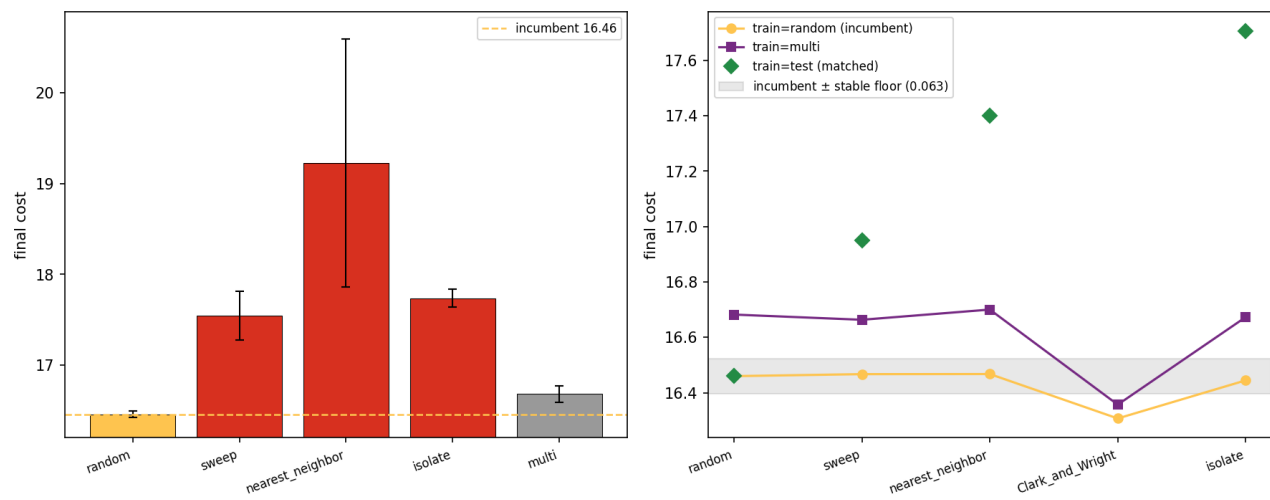


Figure 25: Left: final cost by training construction at the standard random-start deployment — random (gold) is far below the rest, with mixed (gray) a close second. Right: final cost of selected actors across the five test constructions: the random-trained actor (gold) is nearly flat and mostly inside the stable-floor band, the mixed actor (purple) sits uniformly a touch above it, and the matched train/test diagonal (green diamonds) climbs away — ruling out a train/test-matching explanation for the training-construction effect.